4.1 Divisibility 與良序原理

對整數 a, b，若存在整數 k 使 $b = ak$，寫 $a \mid b$。這允許負因數與方向，$a \mid b$ 不是除法結果。對任意整數 m, n，基本性質為：

$$a \mid b,\ a \mid c \Rightarrow a \mid (mb + nc).$$

良序原理（well-ordering principle）：每個非空的正整數子集都有最小元素。它不直接適用於所有非空整數集合，例如全體負整數沒有最小值。

本講義的整除定義允許除數為零：$0 \mid b$ 恰在 $b = 0$ 時成立，所以 $0 \mid 0$ 成立。參考用書則將整除關係的除數限定為非零；比對定理時須注意此慣例差異。除法演算法仍要求除數非零。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.2 Weak induction 的精確形式

若要證 $\forall n \geq n_0, P(n)$：

1. 證 $P(n_0)$。
2. 對任意 $k \geq n_0$，假設 $P(k)$ 並推出 $P(k+1)$。

這只證明所有大於等於 n_0 的後繼鏈都被涵蓋。基底情形（basis）不是形式裝飾；若從錯誤起點跳過，命題就可能失效。

反例警示：對命題「所有整數 $n \geq 2$ 都是偶數」，若遞推步直接寫成由 $2k$ 推出 $2(k+1)$，就只處理了偶數，沒有證明任意固定 k 的蘊涵。

遞推步中的 k 是任取但固定；可多次使用 $P(k)$，也可再依 k 的大小分類。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.2.1 為何 well-ordering 能推出 induction

歸納法不是把「下一個」檢查很多次後相信全部成立。它的有效性可由正整數的良序原理證明。

假設

1. $P(n_0)$ 為真；
2. 對每個 $k \geq n_0$，$P(k) \to P(k+1)$。

令反例集合

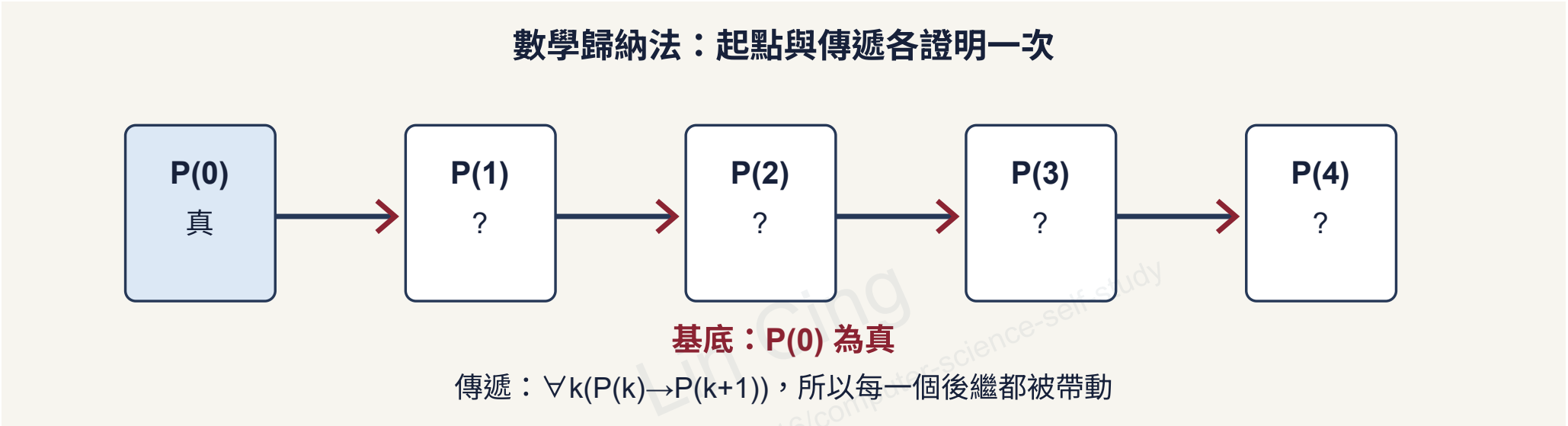
$$S = \{n \in \mathbb{Z} : n \geq n_0 \wedge \neg P(n)\}.$$

若命題不是對所有 $n \geq n_0$ 成立，則 S 非空。固定整數起點 n_0，平移後的 $\{n - n_0 + 1 : n \in S\}$ 是非空正整數集合；由良序原理，S 也有最小元素 m。因為 $P(n_0)$ 為真，所以 $m \neq n_0$。又因 $m \geq n_0$，可得 $m - 1 \geq n_0$。由 m 的最小性，$m - 1 \notin S$，即 $P(m-1)$ 為真。套用遞推步 $P(m-1) \to P(m)$，得到 $P(m)$ 為真，與 $m \in S$ 矛盾。

因此 $S = \varnothing$，命題對所有 $n \geq n_0$ 成立。這個證明也指出基底與遞推步為什麼缺一不可：若少了基底，良序論證無法排除 $m = n_0$ 的失敗；若少了遞推步，無法從一個真命題推到後繼。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

歸納假設與骨牌模型



「假設 $P(k)$」不是假設整個定理已成立。它是對任意固定 k 的條件假設；在遞推步結束時，我們證明的是 implication。

4.2.2 完整歸納例與邊界

先證明對所有 $n \geq 1$：

$$1 + 3 + \cdots + (2n - 1) = n^2.$$

當 $n = 1$ 時，左邊是 1，右邊是 $1^2 = 1$。假設對某個固定的 $k \geq 1$，

$$1 + 3 + \cdots + (2k - 1) = k^2.$$

那麼下一項是 $2(k + 1) - 1 = 2k + 1$，因此

$$\begin{aligned} 1 + 3 + \cdots + (2k - 1) + (2k + 1) &= k^2 + (2k + 1) \\ &= (k + 1)^2. \end{aligned}$$

這正好是 $n = k + 1$ 的結論，所以由 induction 得證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

歸納起點與不等式

再考慮只在較大起點成立的命題：

$$P(n):\quad 4n < n^2 - 7, \qquad n \ge 6.$$

當 $n = 6$，左邊為 24，右邊為 $36 - 7 = 29$，故基底成立。假設 $4k < k^2 - 7$ 對某個 $k \ge 6$ 成立。要證 $4(k+1) < (k+1)^2 - 7$：

$$4(k+1) = 4k + 4 < k^2 - 3.$$

因為 $k \ge 6$，所以 $2k > 3$，從而

$$k^2 - 3 < k^2 + 2k - 6 = (k+1)^2 - 7.$$

合併不等式即完成遞推步。不能把基底改成 $n = 1$：例如 $n = 1$ 時 $4 < -6$ 顯然為假，這也說明命題的起點必須由實際成立範圍決定。

寫歸納證明時可先將目標 $P(k+1)$ 寫在草稿上，再找如何從 $P(k)$ 推過去；正式稿應從假設出發，最後落在目標形式，不能把目標直接假設成已知。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.3 Strong induction 與 well-ordering 轉換

Strong induction 在證 $P(n)$ 時假設所有 $n_0 \le j < n$ 的 $P(j)$。常用於整數分解、圖的去除操作與分治結構。

Weak induction 是 strong induction 的特例。要把 well-ordering 用於全稱命題，可令

$$S = \{n \ge n_0 : \neg P(n)\}.$$

若假設存在反例，良序原理給出最小反例，再導出更小反例或矛盾。

最小反例證法與 contrapositive 是不同包裝；前者需清楚說明集合非空。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.3.1 Weak 與 strong induction 的等價性

Weak induction 與 strong induction 不是兩套不同真假標準。若 ordinary weak induction 有效，則 strong induction 的證明形式也有效。

假設 $P(n_0)$ 成立，且對每個 $n > n_0$，只要

$$P(n_0), P(n_0+1), \dots, P(n-1)$$

都成立，就能推出 $P(n)$。定義輔助命題

$$Q(n): \quad P(n_0) \land P(n_0+1) \land \cdots \land P(n).$$

當 $n = n_0$ 時，$Q(n_0) = P(n_0)$。固定任意 $k \geq n_0$，假設 $Q(k)$ 成立。由 $Q(k)$ 得知 $P(n_0), P(n_0+1), \dots, P(k)$ 全部為真；strong induction 的給定遞推規則可推出 $P(k+1)$，所以 $Q(k+1)$ 成立。Ordinary induction 證明 $Q(n)$ 對所有 $n \geq n_0$ 成立，而 $Q(n) \Rightarrow P(n)$，故 P 也全部成立。

反向地，weak induction 是 strong induction 的特例：若 strong induction 已假設所有較早的命題，當然也可使用最近的 $P(k)$。因此 strong induction 不會多證明某些不存在的命題，它只是允許遞推步使用整個歷史。

強歸納：二階遞迴的逐步證明

完整例：令 $a_0 = 1, a_1 = 2$，且對 $n \geq 2$

$$a_n = 2a_{n-1} + 3a_{n-2}.$$

宣稱

$$a_n = \frac{3^{n+1} + (-1)^n}{4}.$$

先檢查 $n = 0$：$(3 + 1)/4 = 1$；再檢查 $n = 1$：$(9 - 1)/4 = 2$。因此兩個必要 base cases 都成立。

固定 $k \geq 1$，假設公式對所有 $0 \leq j \leq k$ 成立。要推 a_{k+1}，使用 recurrence 與 $k, k - 1$ 的 induction hypotheses：

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

強歸納：二階遞迴的逐步證明 （續）

$$
\begin{aligned}
a_{k+1} &= 2a_k + 3a_{k-1} \\
&= \frac{6 \cdot 3^k + 3 \cdot 3^k + 2(-1)^k - 3(-1)^k}{4} \\
&= \frac{9 \cdot 3^k - (-1)^k}{4} \\
&= \frac{3^{k+2} + (-1)^{k+1}}{4}.
\end{aligned}
$$

最後一式正是 a_{k+1} 的公式，所以 strong induction 完成。兩項 recurrence 需要兩個基底；只檢查 a_0 不足以同時啟動依賴 a_{k-1} 的遞推步。

「強」指的是 induction hypothesis 使用範圍更廣，不是證明力更強的邏輯系統。所有 strong induction 證明都可改寫成對前綴合取命題的 weak induction 證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

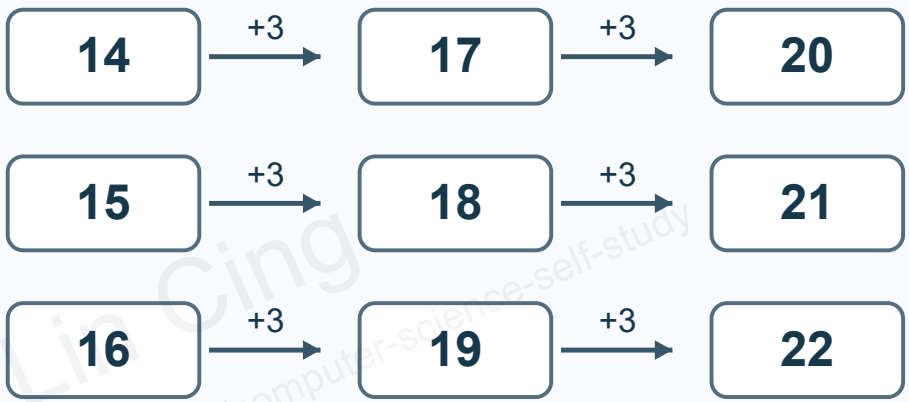
強歸納：三元與八元郵票

三條餘數鏈：每一步加 3

基例：14 = 3 + 3 + 8

基例：15 = 5 × 3

基例：16 = 8 + 8



要證每個整數 $n \geq 14$ 都可由 3 元與 8 元郵票湊出。基例為 $14 = 3 + 3 + 8$、$15 = 5 \cdot 3$、$16 = 8 + 8$。每次加 3 保持可湊出，三個基例分別接住三種餘數。

郵票例：強歸納的遞推步

任取整數 $n \geq 17$，假設 $14, 15, \ldots, n-1$ 均可湊出。因為 $14 \leq n-3 < n$，歸納假設適用於 $n-3$；在其郵票組合加一張 3 元郵票，即湊成 n。

基例覆蓋 $14, 15, 16$，遞推步覆蓋所有 $n \geq 17$。若只驗 14，便無法啟動另外兩條餘數鏈。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.4 Recursive definitions

Recursive definition 由 base cases 與 rule 構成。例如

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2}\ (n \geq 2).$$

存在性與唯一性需一併說明。若每個新值都只依賴更小索引，所有 n 可由有限次迭代得到唯一值。若 recurrence 依賴負索引或沒有足夠 base cases，則未必定義完整序列。

存在唯一性定理：令 $r \geq 1$ 為整數，S 為非空值集合，基底 $a_0, \ldots, a_{r-1} \in S$。函數 f 必須對每個 $n \geq r$ 及每個長度 n 的 S-值前綴都有定義，且結果屬於 S。若

$$a_n = f(n, a_0, \ldots, a_{n-1}) \qquad (n \geq r),$$

則整串 $(a_n)_{n \geq 0}$ 恰有一組解。

由自然數遞迴定理可構造所需無限序列。具體地，基底已唯一指定；每個有限前綴透過全定義的 f 唯一延長一項。不同長度的前綴彼此相容，因此第 n 項可取任一長度超過 n 的前綴之第 n 項，得到整個序列。若另一序列滿足相同基底與規則，對前綴長度歸納可知兩序列逐項相同，故唯一。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

遞迴初值與唯一性

例如 $b_0 = 0, b_1 = 1, b_n = b_{n-1} + b_{n-2}$ 依次得到

$$b_2 = 1,\ b_3 = 2,\ b_4 = 3,\ b_5 = 5,\ b_6 = 8,\ b_7 = 13.$$

若只寫 $c_n = c_{n-1} + c_{n-2}$ 而不給兩個初值，$0, 1, 1, 2, \ldots$、$2, 5, 7, 12, \ldots$ 等不同序列都符合 recurrence，所以不唯一；若寫 $d_n = d_{n+1}$ 且沒有額外邊界，也不能由此唯一決定無窮序列。

「recursive」描述定義方式，不等於方程式一定有唯一實數解；結構歸納與初始條件共同決定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

調和數與 Lucas 數列的起點

調和數以 $H_0 = 0$ 起步，對整數 $n \geq 1$ 定義 $H_n = H_{n-1} + 1/n$。連續展開得到：

n	0	1	2	3	4
H_n	0	1	$3/2$	$11/6$	$25/12$

例如 $H_4 = H_3 + 1/4 = 11/6 + 1/4 = 25/12$。這裡 $H_0 = 0$ 是空和的約定，不把 $1/0$ 當成一項。

Lucas 數列則以 $L_0 = 2, L_1 = 1$ 起步，對 $n \geq 2$ 定義 $L_n = L_{n-1} + L_{n-2}$，因此 $L_2 = 3, L_3 = 4, L_4 = 7$。它與 Fibonacci 數列使用相同遞推式，初值卻不同，所以不能只憑遞推式認定兩者相同。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Eulerian 數：以下降位置分類排列

對整數 $n \geq 1$，令 $A(n,k)$ 為 $1,\dots,n$ 的排列中，恰有 k 個 descent（下降位置）的排列數；descent 指相鄰兩項 $\pi_i > \pi_{i+1}$。本頁採 $A(0,0)=1$；$n \geq 1$ 時若 $k<0$ 或 $k \geq n$，令 $A(n,k)=0$，並令 $A(0,k)=0$（$k \neq 0$）。因此 $A(1,0)=1$。

對 $n \geq 2$ 插入最大元素 n，按插入前排列的下降數分類：

- 原有 k 個下降：插在任一下降之間或末尾，有 $k+1$ 個位置，下降數不變。
- 原有 $k-1$ 個下降：插在開頭或任一上升之間，有 $n-k$ 個位置，下降數增加一個。

刪去最大元素可唯一還原原排列及插入位置，所以這兩類不漏不重，得到

$$A(n,k) = (k+1)A(n-1,k) + (n-k)A(n-1,k-1).$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Eulerian 遞推的三元素例子

由 $A(2,0)=A(2,1)=1$ 和越界為 0，逐項計算：

$$A(3,0)=1,\quad A(3,1)=2(1)+2(1)=4,\quad A(3,2)=1.$$

排列	下降數	排列	下降數
123	0	231	1
132	1	312	1
213	1	321	2

所以 $1+4+1=6=3!$。一般而言，每個排列恰有一個下降數，故分類的列和是 $n!$；這是互斥分類，而非單憑前三列猜測。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

4.5 Recursive algorithm 與結構歸納

Recursive factorial 的 base case 必須回傳明確值，recursive call 的 argument 必須嚴格下降且仍屬合法 domain。證明正確性可按 argument 的大小做 structural induction。

例：若 $\mathrm{bitsum}(0) = 0$，對 $n > 0$：

$$\mathrm{bitsum}(n) = \mathrm{bitsum}(\lfloor n/2 \rfloor) + (n \bmod 2).$$

每步移到非負且更小的整數；indices 形成 well-founded chain，故終止。

對 bitsum 的結果可做更完整的逐步追蹤：

$$\mathrm{bitsum}(45) = 1 + \mathrm{bitsum}(22) = 1 + 0 + \mathrm{bitsum}(11) = \cdots = 4.$$

完整餘數序列為 $1, 0, 1, 1, 0, 1$，與 $45 = (101101)_2$ 中四個 1 相符。終止性由每步 $n \mapsto \lfloor n/2 \rfloor < n$ 得到；正確性則用 strong induction：對 $0 \le j < n$ 假設已等於 binary digit sum，再由第 n 個 recurrence 加上最後一個 binary digit。

終止性與正確性分開證。complexity 由 recursive tree 或 master theorem 分析。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

4.6 Division algorithm

對任意整數 a 與正整數 $d > 0$，存在唯一整數 q, r 使

$$a = qd + r, \qquad 0 \le r < d.$$

存在性：取 $q = \lfloor a/d \rfloor$、$r = a - qd$，即得餘數範圍。

唯一性：若 $qd + r = q'd + r'$，則 $r - r' = (q' - q)d$，且兩餘數差落在 $(-d, d)$，只能是 0。

在 C/C++ 中 `%` 對負數的結果依語言規則，不以一般數學 quotient/remainder 為準。

延伸：$a \bmod d$ 通常定義為 $0 \le r < d$，與實作語言可能不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.6.1 Division algorithm 的良序證明

前面用 floor 直接構造存在性，簡潔但把另一項重要工具藏了起來。以下用良序原理說明為什麼一定能找到最小非負餘數。

固定整數 a 與正整數 d。考慮所有非負差

$$S = \{a - dq : q \in \mathbb{Z},\ a - dq \geq 0\}.$$

S 非空：取 $q_0 = -|a| - 1$。因為 $d \geq 1$，所以

$$a - dq_0 = a + d(|a| + 1) > 0.$$

由 well-ordering principle，S 有最小元素，記為 $r = a - dq$。由 S 的定義，$r \geq 0$。若 $r \geq d$，則

$$r - d = a - dq - d = a - d(q + 1)$$

仍在 S，但比 r 小，矛盾。因此 $0 \leq r < d$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

商餘的唯一性

唯一性方面，若又有 $a = dq' + r'$ 且 $0 \leq r' < d$，則

$$r - r' = d(q' - q).$$

所以左側是 d 的倍數；另一方面

$$-d < r - r' < d.$$

這個區間唯一 d 的倍數是 0，所以 $r = r'$。再由 $d(q' - q) = 0$ 與 $d > 0$ 得 $q' = q$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

商餘的唯一性 （續）

數學 division algorithm

-37 = 8(-5) + 3

q=-5，r=3

條件：0 ≤ r < 8

商與餘式在數學定義下唯一

truncation 與非負餘數

-37 = 8(-4) + (-5)

C/C++ 的 -37 % 8 可為 -5

固定定理要求 0 ≤ r：r += 8，q -= 1

程式規則不等於數學定理的餘式 convention

先固定 quotient/remainder 的定義，再談 code 的行為

證明 existence 與 uniqueness 是兩個方向。只展示一組 q, r 沒有排除第二組；只證唯一性也沒有證明存在。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

4.6.2 進位表示與 two's complement

對非負整數n，重複使用division algorithm，可寫成任意整數base $b \geq 2$ 的展開：

$$n = r_0 + r_1 b + r_2 b^2 + \cdots + r_k b^k, \qquad 0 \leq r_i < b.$$

例如 47 以 3 為 base：

$$\begin{aligned} 47 &= 3 \cdot 15 + 2, \\ 15 &= 3 \cdot 5 + 0, \\ 5 &= 3 \cdot 1 + 2, \\ 1 &= 3 \cdot 0 + 1. \end{aligned}$$

把餘數由下往上讀，得 $47 = (1202)_3$。核驗：

$$1 \cdot 3^3 + 2 \cdot 3^2 + 0 \cdot 3 + 2 = 27 + 18 + 2 = 47.$$

n>0時，每輪quotient由n變floor(n/b)<n，故有限步停在0，回代給存在性，最高位非0。若另有同一n的展開，兩邊mod b給最低位相同，減去該位再除b，逐位重複，故表示唯一。n=0約定寫單一digit0；負整數另加負號，不聲稱全由非負digits的正冪和表示。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

二補數的值域與溢位

固定 n bits時，two's complement 用最高 bit 表示符號。以 8 bits 表示 -6：先寫 $6 = (00000110)_2$，逐位反向得到 $(11111001)_2$，再加 1 得

$$-6 = (11111010)_2.$$

更精確地，w bits的value為 $-b_{w-1}2^{w-1} + \sum_{i=0}^{w-2} b_i 2^i$，範圍 $[-2^{w-1}, 2^{w-1} - 1]$。它與unsigned bit pattern U相差最高位為1時的2^w；−6在8 bits的U=2⁸−6=250，依此解讀回−6。負值最高位不是單純貼上的正負標籤，而有負權重。

把 $(11111010)_2$ 再取反向加一，會回到 00000110。固定寬度的 signed addition 可容許棄掉最左進位，但若兩個同號數相加後符號改變，仍是 overflow。例如 8-bit 下 $127 + 1$ 得到 bit pattern 10000000，解釋為 -128，所以不能把 wire-level 的 pattern 與數學整數值混為一談。

Base expansion 由 division 的「由低位到高位」得到；two's complement 的固定寬度也決定可表示範圍，兩者都必須寫出 convention。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.7 質數、質元素與不可約元素的完整定義

整數 p>1 是質數，若正因數只有 1 與 p。1 是單位，不是質數；0 也不是質數。

在整域 R 中，質元素 p 必須先滿足 p≠0 且 p 不是單位，再要求

$$p \mid ab \Rightarrow p \mid a \text{ 或 } p \mid b \qquad (a, b \in R).$$

單位 u 是具有乘法逆元的元素。不可約元素 p 也先要求非零、非單位，再要求 p=ab 時 a 或 b 至少一個為單位。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

質元素條件與反例

類型	是否可能滿足單獨的整除蘊涵	是否為質元素
0	在整域中會滿足，因 ab=0 強迫 a=0 或 b=0	否，缺少非零條件
單位，如 Z 中 ±1	會滿足，因單位整除所有元素	否，缺少非單位條件
Z 中 2	滿足 Euclid 引理	是，也不可約
Z 中 4	4整除2・2，卻不整除任一2	否，且可約為2・2

質元素必不可約：若 p=ab 且 p整除a，寫a=pc，消去非零p得1=cb，故b是單位；另一方向對稱。反向在 Z 成立，但在一般整域未必成立，不能把兩個定義合併。

對正整數 n 的試除，只需檢查至 sqrt(n)：若 n=ab 且 1<a≤b，必有 a≤sqrt(n)。

4.7.1 質數的基本定理與反例

先證明每個合數都有質因數。令 $n > 1$ 為 composite，考慮 n 的所有非平凡 divisor d（$1 < d < n$）。這個集合非空，由 well-ordering principle 取最小 p。若 p 仍可分解為 $p = ab$ 且 $1 < a < p$，則 $a \mid n$ 且 $a < n$，與 p 的最小性矛盾；所以 p 必為 prime。

更進一步，必存在 prime $p \leq \sqrt{n}$ 使 $p \mid n$。把 $n = pk$，因 p 是「最小的非平凡 factor」，所以 $p \leq k$。左右同乘 p 得 $p^2 \leq pk = n$，因此 $p \leq \sqrt{n}$。

這個界也解釋試除法的停止條件：若要判斷 $n > 1$ 是否 prime，只需檢查 $2 \leq d \leq \sqrt{n}$ 是否存在 divisor。若有 divisor $ab = n$ 且 $1 < a \leq b$，必有 $a \leq \sqrt{n}$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

無限多質數的證明

Euclid 的無限質數證明假設只有有限個 primes

$$p_1, p_2, \ldots, p_t.$$

令

$$N = p_1 p_2 \cdots p_t + 1.$$

因為 $N > 1$，若 N 本身是質數，就取 $p = N$；若 N 是合數，則由上段結果取其質因數 p。假設中的每個 prime 都整除乘積 $p_1 \cdots p_t$，所以每個 p_i 除 N 的餘數都是 1，不可能整除 N。但 N 的 prime divisor 必是某個 p_i，矛盾。故質數有無限多個。

反例邊界：1 不是 prime，否則 FTA 的唯一性會被任意多個 1 破壞；0 也不是 prime，且通常不是 prime element 的候選。對 $n = 2$，判準「只檢查到 $\sqrt{n}$」仍成立，但沒有 candidate $d \geq 2$，因此 2 為 prime。對 $n = 1$，判準不適用，因題目假設 $n > 1$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.8 GCD 與 Euclidean algorithm

對不全為 0 的整數 a, b，$\gcd(a, b)$ 是同時整除二者的最大正整數。對 $b > 0$，

$$\gcd(a, b) = \gcd(b, a \bmod b).$$

證明關鍵是：$a = qb + r$ 時，a, b 的公因數集合等於 (b, r) 的公因數集合。以 division algorithm 反覆下降，餘數最終成 0；前一個正餘數即 gcd。

任一輸入為0時直接返回另一者的絕對值，只需常數步；兩者非0並正規化為正數後，每兩步餘數至少減半，需 $O(\log \min(|a|, |b|) + 1)$ 次除法。此為除法次數，不是大整數bit complexity。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.8.1 gcd 的線性組合刻畫與演算法證明

先證明一個比最大公因數定義更深的性質。對不全為 0 的整數 a, b，令

$$S = \{ax + by : x, y \in \mathbb{Z},\ ax + by > 0\}.$$

若a≠0，a與−a至少一個為正而屬S；若a=0，由不全為0知b≠0，則b與−b至少一個為正而屬S。所以S非空。由 well-ordering principle，令 d 為 S 的最小元素。存在整數 x_0, y_0 使

$$d = ax_0 + by_0.$$

使用 division algorithm 寫 $a = dq + r$，其中 $0 \le r < d$。則

$$\begin{aligned} r &= a - dq \\ &= a - (ax_0 + by_0)q \\ &= a(1 - x_0 q) + b(-y_0 q). \end{aligned}$$

如果 $r > 0$，它就會是 S 中比 d 更小的元素，矛盾。因此 $r = 0$，即 $d \mid a$。同理可得 $d \mid b$。

若 $c > 0$ 是任意公因數，則 $c \mid a, c \mid b$，所以 $c \mid ax_0 + by_0 = d$，因而 $c \le d$。因此 $d = \gcd(a, b)$，同時證得 Bézout 表示存在。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

輾轉相除的不變量與回代

Euclidean algorithm 的正確性來自：若 $a = qb + r$，則

$$d \mid a \text{ 且 } d \mid b \iff d \mid b \text{ 且 } d \mid r.$$

兩邊方向的證明都只用 $r = a - qb$ 與 $a = qb + r$。因此每一步不改變「共同因數集合」，最終 $r = 0$ 時 gcd 就是最後非零餘數。餘數每次嚴格小於前一個除數，所以必在有限步歸零。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

輾轉相除的不變量與回代 （續）

輾轉相除與 Bézout 回代

除法步驟

1. $1071 = 2 \times 462 + 147$
2. $462 = 3 \times 147 + 21$
3. $147 = 7 \times 21 + 0$

餘數依序下降: 147 → 21 → 0

共同因數集合不變

C(a,b) = a 與 b 的正共同因數集合

C(1071, 462) = C(462, 147)
= C(147, 21) = C(21, 0)

因 r = a − qb，兩組共同因數相同

餘數為 0：最後非零餘數 = gcd = 21

反向回代（將 147 換成 1071 − 2 × 462）

$$21 = 462 - 3 \times 147 \rightarrow 21 = 7 \times 462 - 3 \times 1071$$

係數：1071 × (−3) + 462 × 7 = 21

d 是「最小正線性組合」這件事不能用來跳過最大性證明。還要處理兩個方向：d 是公因數，以及所有公因數都整除 d。

4.9 Bézout identity

對不全為 0 的整數 a, b，存在整數 s, t 使

$$\gcd(a, b) = sa + tb.$$

Extended Euclidean algorithm 不僅求 gcd，也保留整數係數。逆轉歐幾里得步驟可證存在性；不要求 s, t 唯一，且它們可能為負數。

例：$\gcd(84, 30) = 6$。由 $84 = 2 \cdot 30 + 24$、$30 = 1 \cdot 24 + 6$，得

$$6 = 30 - 24 = 30 - (84 - 2 \cdot 30) = 3 \cdot 30 - 84.$$

例如，$84x + 30y = 12$ 因 6 整除 12 而有解。由上式

$$-84 + 3 \cdot 30 = 6,$$

左右乘 2 得

$$-2 \cdot 84 + 6 \cdot 30 = 12.$$

所以可行的一組是 $(x, y) = (-2, 6)$，代回確實得到 12。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

線性不定方程的可解性

Bézout 係數不唯一：若 $d = sa + tb$，令 $g = \gcd(a, b)$，則對每個整數 k，

$$d = (s + k\,b/g)a + (t - k\,a/g)b.$$

線性 Diophantine equation對不全為0的整數a,b與整數c，方程式

$$ax + by = c$$

有整數解，若且唯若 $\gcd(a, b) \mid c$。

a=b=0須另處理：c=0時所有整數對都解；c≠0時無解，不引用此處未定義的gcd(0,0)。

若 $g = \gcd(a, b)$ 整除 c，寫 $c = g\ell$。由 Bézout 得 $g = sa + tb$，乘 ℓ 得 $c = (s\ell)a + (t\ell)b$，所以有解。反之，若有解，任何公因數都整除左側，因此也整除 c，特別 $\gcd(a, b) \mid c$。

第 14 章會用 Bézout identity 判定 modulo n 下的乘法反元素。

4.9.1 gcd、lcm 與公共倍數

先給出 lcm 的精確定義：對正整數 a, b，$\operatorname{lcm}(a, b)$ 是同時為 a, b 倍數的最小正整數。

對正整數 a, b，有

$$\gcd(a, b)\operatorname{lcm}(a, b) = ab.$$

令 $g = \gcd(a, b)$，寫 $a = gA, b = gB$，其中 $\gcd(A, B) = 1$。候選值 $L = gAB = aB = bA$ 同時是 a, b 的倍數。

現在令正整數 M 也是公共倍數，寫 $M = au = bv$。代入 $a = gA, b = gB$，得 $gAu = gBv$，故 $Au = Bv$。

因為 $\gcd(A, B) = 1$，Bézout給sA+tB=1；乘v後兩項皆被A整除（第二項因Bv=Au），所以A|v。設 $v = Aw$，則

$$M = bv = gB(Aw) = gABw = Lw.$$

所以每個正公共倍數都能被 L 整除，且 $L \le M$。L 是最小正公共倍數。

因此 $\gcd(a, b)\operatorname{lcm}(a, b) = g \cdot gAB = g^2AB = ab$。

最小公倍數數例與範圍

以 $\gcd(168, 456) = 24$ 為例：

$$\operatorname{lcm}(168, 456) = \frac{168 \cdot 456}{24} = 3192.$$

最小性檢查可看分解 $168 = 24 \cdot 7$、$456 = 24 \cdot 19$，且 $\gcd(7, 19) = 1$，所以 $24 \cdot 7 \cdot 19 = 3192$；任何公共倍數都必須包含 7 與 19 的倍數。

$\gcd(a, b) \cdot \operatorname{lcm}(a, b) = ab$ 只對指定正整數版本直接成立；若寫入一般整數、負數或 0，要先逐項重定符號與 domain。特別是 $\gcd(0, n) = |n|$，而 $\operatorname{lcm}(0, n)$ 也需明示慣例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.9.2 Prime 與 irreducible 的差別

在一般含單位元的交換整域中，兩詞必須先精確定義。非零、非 unit 的 p 是 prime element，若

$$p \mid ab \Rightarrow p \mid a \text{ 或 } p \mid b.$$

非零、非 unit 的 p 是 irreducible，若

$$p = ab \Rightarrow a \text{ 是 unit 或 } b \text{ 是 unit}.$$

Prime 必 irreducible。 若 prime $p = ab$，由 $p \mid ab$ 與 prime 性質，$p \mid a$ 或 $p \mid b$。在第一種情形寫 $a = pc$，則 $p = pbc$。因為 $p \neq 0$ 且所在環是 integral domain，可消去 p，得到 $1 = bc$，所以 b 是 unit。另一情形同理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

整數中的不可約元素與質元素

在 $\mathbb{Z}$ 中，irreducible 也必 prime。 任取不可約整數 p，令 $q = |p| > 1$。因 p 與 q 只差單位 ± 1，q 也不可約，且對任意整數 z，$p \mid z$ 當且僅當 $q \mid z$。因此只需證明正的不可約整數具有 prime 性質；以下將它記為 $p > 1$，並設 $p \mid ab$。若 $p \nmid a$，則 $\gcd(p, a) \neq p$。因 p 的 positive divisors 只有 1 與 p，必有 $\gcd(p, a) = 1$。由 Bézout，

$$up + va = 1.$$

乘 b 得 $upb + vab = b$。兩項都被 p 整除，因此 $p \mid b$。故 p 是 prime。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一般整域中的反例

但在一般整域兩者可不等價。例如

$$R = \mathbb{Z}[\sqrt{-5}] = \{a + b\sqrt{-5} : a, b \in \mathbb{Z}\}.$$

定norm $N(a + b\sqrt{-5}) = a^2 + 5b^2$。直接展開乘法得N(uv)=N(u)N(v)，非零元素的norm為正整數；norm1恰是± 1，正是units（unit的norm須有整數乘法inverse）。

若3=uv，則N(u)N(v)=N(3)=9。若兩者均非unit，norm只能各為3；但$a^2+5b^2=3$無整數解：$b \neq 0$時至少5，$b=0$時要求$a^2=3$。因此至少一因子的norm1而為unit，證明3 irreducible。不能說norm小於9都為units，例如2的norm4、$\sqrt{-5}$的norm5都是反例。然而

$$(2 + \sqrt{-5})(2 - \sqrt{-5}) = 9,$$

所以3整除乘積9；若3整除$2 \pm \sqrt{-5}$，對比整數係數會要求3整除2與± 1，不可能。因此3不整除任一factor，在此環不是prime。Z中的等價性不能搬到所有整域。

Prime 與 irreducible 的定義都包含「非零、非 unit」前提。單獨的質整除蘊涵會把 0 與 units 也算進來，所以這些限制不可省略。

4.10 Fundamental Theorem of Arithmetic

每個整數 $n \geq 2$ 都可表示為 primes 的乘積；忽略順序後，表示法唯一。存在性用 strong induction：若 n 非 prime，寫 $n = ab$ 且 $1 < a, b < n$，對 a, b 遞迴分解。

唯一性可用 Euclid's lemma：若 prime $p \mid ab$，則 $p \mid a$ 或 $p \mid b$。此引理可先由 Bézout 證明而不依賴唯一分解：若 $p \nmid a$，則 $\gcd(p, a) = 1$，有 $up + va = 1$。乘以 b，由 $p \mid ab$ 得 $p \mid b$。比較兩個質數乘積，一側的質數整除另一側某質因數，兩質數因而相等；消去後反覆進行。若一側尚有剩餘質因數，其乘積必須等於 1，不可能。

把存在性與唯一性完整展開如下。

存在性。 基底 $n = 2$ 本身是 prime，可視為一個 prime 的乘積。任取 $n > 2$，假設所有 $2 \leq j < n$ 都能分解。若 n 是 prime，已完成；若 n 是 composite，則存在 $1 < a < n$ 使 $a \mid n$。寫 $n = ab$，其中 $1 < a, b < n$。由 strong induction，a, b 各有 prime factorization，把它們的 factors 合併就得到 n。由 strong induction，所有 $n \geq 2$ 都有 prime factorization。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Euclid 引理與唯一性

Euclid 引理。 若 prime $p \mid ab$ 且 $p \nmid a$，則 $\gcd(p, a) = 1$。由 Bézout 存在 u, v 使

$$up + va = 1.$$

兩邊乘 b：

$$upb + vab = b.$$

左側第一項被 p 整除；第二項因 $p \mid ab$ 也被 p 整除。故 $p \mid b$。所以 $p \mid a$ 或 $p \mid b$。

推到任意長乘積。 對 factor 數 m 使用歸納。$m = 1$ 顯然。若

$$p \mid a_1 a_2 \cdots a_{m+1},$$

先用 Euclid 引理把乘積切成 $a_1 \cdots a_m$ 與 a_{m+1}。若 $p \mid a_{m+1}$ 已完成；否則 $p \mid a_1 \cdots a_m$，套用歸納假設，可知 p 整除其中至少一個 a_i。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

質因數分解的唯一性

唯一性。 假設

$$n = p_1 p_2 \cdots p_r = q_1 q_2 \cdots q_s$$

是兩個 prime factorizations。p_1 整除左側，因此也整除右側。由 generalized Euclid lemma，存在 j 使 $p_1 \mid q_j$。兩者都是 prime，所以 $p_1 = q_j$。重排右側 factors 並消去此共同 prime，得到較短的兩組相等乘積。反覆消去後，如果一側先耗盡而另一側仍有 prime，該剩餘乘積會大於 1，不可能等於 1。因此兩側的 primes 可用一對一方式配對，且次數相同；忽略順序後唯一。

在唯一分解整域中，分解的唯一性是指相差因子順序與單位倍數；一般整域不一定具有這種唯一性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.10.1 標準分解式、因數計數與 perfect square

把 FTA 的唯一因子重數寫成

$$n = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_r^{\alpha_r}, \qquad \alpha_i \geq 1.$$

n 的每個正因數都可唯一寫成

$$d = p_1^{\beta_1} \cdots p_r^{\beta_r}, \qquad 0 \leq \beta_i \leq \alpha_i.$$

因此正因數個數為

$$\tau(n) = (\alpha_1 + 1)(\alpha_2 + 1) \cdots (\alpha_r + 1).$$

這是 multiplication principle 與 FTA 的聯合應用：每個 prime 的 exponent 可獨立選取，且選法唯一。

d 是 perfect square，若且唯若所有 β_i 都是 even。因此 perfect-square divisors 的個數是

$$\prod_{i=1}^{r} \left(\left\lfloor \frac{\alpha_i}{2} \right\rfloor + 1 \right).$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

因數計數與完全平方數例

例：

$$360 = 2^3 \cdot 3^2 \cdot 5.$$

所以

$$\tau(360) = (3+1)(2+1)(1+1) = 24.$$

Perfect-square divisors 對應 2 的 exponent $0, 2$、3 的 exponent $0, 2$、5 的 exponent 0，共 $2 \cdot 2 \cdot 1 = 4$ 個，分別是

$$1,\ 4,\ 9,\ 36.$$

若 n 是 perfect square，寫 $n = m^2$，則其在 FTA 中的每個 prime exponent 都是 even。反向由同一個 standard form 直接平方回去。

「有 odd number of positive divisors 若且唯若是 perfect square」可由 exponent choices 證明：乘積 $(\alpha_i + 1)$ 為 odd，若且唯若每個 α_i 都 even。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

100! 的尾零：逐層計數

100! 的 5 因子分層

5^1	$\lfloor 100 / 5 \rfloor = 20$	5, 10, …, 100
5^2	$\lfloor 100 / 25 \rfloor = 4$	25, 50, 75, 100
5^3	$\lfloor 100 / 125 \rfloor = 0$	125 > 100

每個 5 的倍數提供一個因子 5，每個 25 的倍數再提供一個，故 $v_5(100!) = 20 + 4 + 0 = 24$。50 個偶數至少提供 50 個因子 2，足以配對；尾零恰為 **24 個**。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

4.11 例：proof defect 與 boundary

考量命題「每個 $n \geq 2$ 都能寫成兩個大於 1 的整數乘積」。所有質數都使命題失敗，故命題為假。若改成「若 n 非質數，則可如此表示」，先要精確定義合數（composite）。

用強歸納證明分解時，不能在基底情形尚未列出 $n = 2$ 前就呼叫 $P(1)$。缺少基底是關鍵錯誤。

證明中每個遞迴引用都要落在已建立或嚴格更小的定義域內。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.12 本章檢查與練習

檢查依序為：量化起點、基底、下降量、除法唯一性、gcd 不變量與質因數分解唯一性的前提。完成 本章練習題 PDF；逐題推導見 本章詳解 PDF。

參考資料與署名

Lin Cing，《離散數學》手寫筆記。 Ralph P. Grimaldi, Discrete and Combinatorial Mathematics, 5th ed., Ch. 4.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study